## Nombre del proyecto

Clasificación automática de normativa sobre inteligencia artificial en la educación superior de América Latina y el Caribe: hacia un Índice de Madurez Normativa (IMN) escalable

**Entregable: Análisis comparativo de algoritmos de IA — Semana 2**

| Profesor | Materia | Fecha de Inicio | Fecha de Finalización |
|---|---|---|---|
| Gladys María Villegas Rugel | Proyecto Integrador | 8 de septiembre de 2026 | 25 de octubre de 2026 |

## Resumen del Proyecto

Comparación de ocho técnicas de IA aplicables a la clasificación de normativa sobre IA en educación superior: regresión logística y SVM sobre TF-IDF, Random Forest, Gradient Boosting, redes neuronales, transformers afinados, LLM few-shot y, como extra, agentes basados en LLM. Evalúa ventajas, desventajas, complejidad computacional, casos de uso y evidencia, y cierra con una matriz de decisión ponderada que fundamenta la Ficha de Decisión Técnica.

### Responsables del proyecto

| Nombre | Rol | Tareas |
|---|---|---|
| Jorge Javier Maldonado Mahauad | Líder de las Fases 1, 2 y 4. Enlace con GENIAL. | Problema y estado del arte; depuración del corpus, variable idioma y muestreo; doble codificación a ciegas y cálculo de κ. |
| Melissa Figallo | Líder de las Fases 3 y 5. | Rama clásica y rama LLM few-shot con comparación pareada; ética, agregación al IMN, documentación y pitch. |

# Análisis comparativo de algoritmos de IA para la clasificación de normativa

**Proyecto Integrador · Semana 2 · Grupo 9** — Jorge Javier Maldonado Mahauad · Melissa Figallo. Documento de respaldo de la Ficha de Decisión Técnica y Arquitectura. Compara ocho técnicas de IA aplicables al problema del proyecto —siete familias predictivas y, como extra, un enfoque basado en agentes— con los criterios de selección revisados en la tutoría de la semana.

## 1. El problema que las técnicas deben resolver

Antes de comparar técnicas conviene fijar con precisión qué se les pide, porque los criterios de selección —tipo de problema, datos y recursos disponibles, interpretabilidad requerida— no se aplican en abstracto sino sobre una tarea concreta. La Tabla 1 resume la caracterización del problema con los hallazgos del Análisis Exploratorio de Datos (EDA) entregado esta misma semana.

**Tabla 1.** Caracterización del problema según los criterios de selección de técnicas.

| Criterio | Valor en este proyecto | Implicación para la selección |
|---|---|---|
| **Tipo de tarea** | Clasificación supervisada multiclase (5 dimensiones del framework GENIAL), etiqueta única por norma | Descarta regresión, agrupamiento y detección de anomalías; exige métricas por clase (F1 macro) por el desbalance 4.7:1 |
| **Tipo de dato** | Texto no estructurado y corto: título de la norma + observaciones (mediana 44 palabras, máximo 274); sin variables numéricas nativas | Las técnicas «tabulares» (Random Forest, Gradient Boosting) solo aplican sobre una representación vectorial del texto; las de secuencia y los modelos de lenguaje aplican de forma nativa |
| **Volumen de datos etiquetados** | 813 normas; dos clases con menos de 75 casos | Insuficiente para entrenar redes profundas desde cero o afinar transformers de forma robusta; suficiente para modelos lineales y para ensamblados de árboles |
| **Idioma** | Títulos en español (712), portugués (76) e inglés (25); observaciones en español casi siempre | El multilingüismo es un efecto de segundo orden, pero existe: las técnicas con representación multilingüe nativa tienen ventaja en los títulos |
| **Modo de operación** | Por lotes (recálculo del índice tras incorporar normas), no en tiempo real | La latencia de inferencia no es restrictiva; el costo por pasada completa sí se contabiliza |
| **Recursos** | Dos portátiles sin GPU; API de un LLM con presupuesto acotado; 7 semanas; 16 h-persona/semana | Penaliza entrenamientos que requieran GPU o búsqueda extensa de hiperparámetros |
| **Interpretabilidad requerida** | Alta: los codificadores de GENIAL exigen saber por qué se propone cada dimensión; el índice compara países y debe ser auditable | Favorece modelos con pesos inspeccionables o con explicación textual; penaliza cajas negras sin mecanismo de explicación |
| **Referente de calidad** | Etiqueta de un único codificador; fiabilidad intercodificador no medida | Ningún modelo puede superar la fiabilidad del estándar; la técnica debe permitir comparar su acuerdo con el humano (κ) y emitir una alternativa (top-2) |

El mapa «Choosing the right estimator» de scikit-learn [1] conduce, para más de 50 muestras, predicción de una categoría, datos etiquetados, menos de 100 000 muestras y datos de texto, a la familia de clasificadores lineales (*LinearSVC*, *Naive Bayes*) como primer intento, y solo si fallan a ensamblados y *kernel approximation*. La comparación que sigue toma ese punto de partida y lo contrasta con las alternativas que la literatura del dominio legal y de política pública considera hoy competitivas.

## 2. Criterios de comparación

Cada técnica se describe con la misma estructura: cómo se aplicaría al problema, ventajas, desventajas, complejidad computacional (entrenamiento e inferencia; $n$ = normas, $d$ = dimensión de la representación, $T$ = árboles o iteraciones, $L$ = longitud de la secuencia en tokens), casos de uso típicos con evidencia publicada y un veredicto para este proyecto. La sección 4 consolida la comparación en una tabla y la sección 5 la convierte en una matriz de decisión ponderada.

## 3. Fichas comparativas

### 3.1 Regresión Logística sobre TF-IDF (modelo lineal)

| | |
|---|---|
| **Cómo se aplicaría** | Título + observaciones → TF-IDF de 1-2 gramas (≈5 000 características, *sublinear_tf*) → Regresión Logística multinomial con *class_weight='balanced'* y regularización L2; validación cruzada estratificada de 5 particiones; *predict_proba* para el umbral de confianza y el top-2. |
| **Ventajas** | Devuelve probabilidades razonablemente calibradas sin pasos adicionales [2], lo que habilita el enrutamiento por confianza. Pesos por término y clase inspeccionables: se puede mostrar a los codificadores qué palabras empujan cada dimensión. Entrena en segundos, sin GPU, sin dependencias externas. Referente estándar de clasificación de texto corto [3]. |
| **Desventajas** | Opera sobre formas superficiales: no captura orden, negación ni sinonimia, y no transfiere entre idiomas. Techo bajo en clases temáticamente heterogéneas (Técnico). Sensible a la pista léxica de la etiqueta detectada en el EDA. |
| **Complejidad** | Entrenamiento $O(n \cdot d \cdot k)$ por iteración del optimizador ($k$ = clases), convergencia en decenas de iteraciones; inferencia $O(d \cdot k)$ por norma; memoria $O(d \cdot k)$. En este corpus: segundos. |
| **Casos de uso típicos** | Clasificación temática de documentos, análisis de sentimiento, *triage* de textos cortos; línea base en LexGLUE y en clasificación legal coreana (≈85 % frente a 99 % de un BERT afinado) [4], [5]. |
| **Veredicto** | **Seleccionada como rama clásica y línea base obligada. Sin ella no hay contra qué comparar el LLM ni forma de justificar el uso de IA frente a las líneas base sin IA.** |

### 3.2 Máquinas de vectores de soporte lineales (LinearSVC)

| | |
|---|---|
| **Cómo se aplicaría** | Misma representación TF-IDF; *LinearSVC* uno-contra-resto con *class_weight='balanced'* y $C$ ajustado por validación cruzada. Probabilidades solo mediante calibración posterior (Platt o isotónica). |
| **Ventajas** | Históricamente la mejor técnica clásica para texto de alta dimensión y disperso [6], [7]; margen máximo robusto al sobreajuste en $d \gg n$; entrenamiento en segundos; coeficientes inspeccionables como en la regresión logística. |
| **Desventajas** | No produce probabilidades de forma nativa: el umbral de confianza exigiría un paso de calibración con datos que en este corpus son escasos. Misma ceguera léxica y monolingüe que cualquier bolsa de palabras. Con núcleos no lineales el costo crece a $O(n^2)$-$O(n^3)$ y se pierde la interpretabilidad. |
| **Complejidad** | Entrenamiento $O(n \cdot d)$ por iteración (*liblinear*), inferencia $O(d \cdot k)$; núcleos RBF: $O(n^2 \cdot d)$ a $O(n^3)$. |
| **Casos de uso típicos** | Categorización de noticias, filtrado de spam, clasificación de documentos jurídicos con miles de características; línea base fuerte en *benchmarks* legales [4]. |
| **Veredicto** | Alternativa casi equivalente a la regresión logística. Se descarta como rama principal solo por la necesidad de probabilidades calibradas para el enrutamiento; se conserva como *ablación* para comprobar que el resultado de la rama clásica no depende del clasificador lineal elegido. |

### 3.3 Random Forest

| | |
|---|---|
| **Cómo se aplicaría** | TF-IDF (o su reducción con SVD truncado a 200-300 componentes) → bosque de 300-500 árboles con *class_weight='balanced_subsample'*; importancia de variables por permutación. |
| **Ventajas** | Robusto a hiperparámetros y a valores atípicos; captura interacciones no lineales entre términos; probabilidades por votación; importancia de variables agregada [8]. |
| **Desventajas** | Diseñado para datos tabulares densos: sobre matrices dispersas de miles de términos y 813 ejemplos, cada árbol ve pocas |

| | |
|---|---|
| | características relevantes por partición y el bosque tiende a rendir por debajo de los modelos lineales en texto [3]. Interpretabilidad global (importancias) pero no local ni por clase. Modelos pesados en memoria. |
| **Complejidad** | Entrenamiento O($T \cdot n \cdot log\ n \cdot \sqrt{d}$), inferencia O($T \cdot profundidad$) por norma; memoria proporcional a $T$ × nodos. En este corpus: minutos. |
| **Casos de uso típicos** | Datos tabulares heterogéneos (crédito, salud, sensores); en texto, sobre representaciones densas reducidas o embeddings, no sobre TF-IDF crudo. |
| **Veredicto** | No seleccionada. Su fortaleza —interacciones no lineales en datos tabulares— no corresponde al problema, y su interpretabilidad es inferior a la del modelo lineal para explicar una dimensión a un codificador. |

## 3.4 Gradient Boosting (XGBoost / LightGBM)

| | |
|---|---|
| **Cómo se aplicaría** | Igual representación que Random Forest; *LightGBM* multiclase con *is_unbalance* o pesos por clase, *early stopping* por validación cruzada. |
| **Ventajas** | Estado del arte en datos tabulares [9], [10]; LightGBM maneja matrices dispersas y entrena rápido; probabilidades directas; buen control del sobreajuste con *early stopping*; importancias y valores SHAP para explicación. |
| **Desventajas** | Muchos hiperparámetros sensibles (tasa de aprendizaje, hojas, regularización) que con 813 ejemplos se ajustan con alta varianza; en texto disperso rara vez supera a un lineal bien regularizado y sí lo iguala en costo de explicación. Riesgo de sobreajuste con clases de 63 casos. |
| **Complejidad** | Entrenamiento O($T \cdot n \cdot d$) con histogramas (LightGBM reduce $d$ efectivo); inferencia O($T \cdot profundidad$); memoria moderada. |
| **Casos de uso típicos** | Riesgo crediticio, predicción de demanda, competiciones tabulares; en texto, como *meta-modelo* sobre características derivadas (longitud, probabilidades de otros modelos). |
| **Veredicto** | No seleccionada como clasificador principal. Queda anotada como opción para un *meta-clasificador* de enrutamiento (combinar probabilidad, idioma, ámbito y país para decidir si una norma va a revisión) si el umbral simple resultara insuficiente. |

## 3.5 Redes neuronales: fastText y CNN sobre embeddings

| | |
|---|---|
| **Cómo se aplicaría** | Embeddings de palabras (preentrenados multilingües o aprendidos) → promedio (fastText) o convoluciones 1-D con *max-pooling* (CNN de Kim) → capa densa softmax de 5 salidas; *dropout* y parada temprana. |
| **Ventajas** | Capturan composición local (n-gramas semánticos) y sinonimia a través de los embeddings; fastText entrena en segundos en CPU [11]; la CNN de sentencias es un referente sólido de clasificación de texto corto [12]; embeddings multilingües alineados mitigan el problema del idioma. |
| **Desventajas** | Con 813 ejemplos y 5 clases la varianza de entrenamiento es alta: cientos de miles de parámetros para menos de mil ejemplos, incluso con embeddings congelados. Caja negra parcial: explicación solo por métodos *post hoc* (saliencia). Requieren ingeniería de entrenamiento (semillas, épocas, *dropout*) que consume tiempo del cronograma. |
| **Complejidad** | Entrenamiento O(épocas · $n \cdot L \cdot d \cdot f$) ($f$ = filtros); inferencia O($L \cdot d \cdot f$) por norma; CPU suficiente para fastText, lenta pero viable para una CNN pequeña. |
| **Casos de uso típicos** | Clasificación de reseñas, detección de intención en chatbots, etiquetado temático con corpus de decenas de miles de ejemplos. |
| **Veredicto** | No seleccionada en este ciclo: el volumen de datos no justifica la varianza adicional y la interpretabilidad se degrada sin ganancia clara sobre el lineal. Candidata natural si el corpus supera varios miles de normas. |

## 3.6 Transformers afinados (BERT multilingüe, BETO, Legal-BERT, XLM-R)

| | |
|---|---|
| **Cómo se aplicaría** | Tokenización *subword* → codificador preentrenado (BETO para español [13], XLM-RoBERTa para cobertura multilingüe [14], Legal-BERT para dominio legal en inglés [15]) → cabeza de clasificación afinada sobre las 813 normas, 3-5 épocas, con validación cruzada. |
| **Ventajas** | Mejor desempeño documentado en clasificación legal: los modelos preentrenados en dominio superan a los genéricos en las siete tareas de LexGLUE [4] y alcanzan 99 % frente al 85 % del TF-IDF en Lee (2025) [5]. Capturan contexto, negación y |

| | |
|---|---|
| | sinonimia; XLM-R transfiere entre idiomas de forma nativa [14], [16]. |
| **Desventajas** | Requieren del orden de miles de ejemplos etiquetados para afinar de forma estable; con dos clases de menos de 75 casos el riesgo de sobreajuste y de varianza entre semillas es alto. Afinado en GPU (horas) o CPU (muy lento); acceso a repositorios de modelos no garantizado en el entorno. Interpretabilidad baja: atención y saliencia no equivalen a una justificación para el codificador. Descartan la comparación de eficiencia de datos que es el núcleo del proyecto. |
| **Complejidad** | Autoatención O($L^2 \cdot d$) por capa y secuencia [17], [18]; afinado O(épocas · $n \cdot L^2 \cdot d \cdot$ *capas*); inferencia decenas de milisegundos por norma en GPU, cientos en CPU; memoria de cientos de MB a GB. |
| **Casos de uso típicos** | Clasificación legal y de política con corpus grandes (MultiEURLEX, LexGLUE), NER, QA, análisis de sentimiento a escala [4], [16]. |
| **Veredicto** | No seleccionada en este ciclo por volumen de datos, cómputo e interpretabilidad; declarada explícitamente como trabajo futuro para cuando el corpus crezca. Es la técnica que un observatorio con 5 000 normas debería considerar. |

## 3.7 Modelos de lenguaje de gran escala en modo few-shot (aprendizaje en contexto)

| | |
|---|---|
| **Cómo se aplicaría** | *Prompt* con las definiciones de las cinco dimensiones del framework GENIAL y 15 ejemplos codificados (3 por dimensión) → la norma en su idioma original → salida estructurada con primera y segunda dimensión; temperatura 0, tres ejecuciones para medir estabilidad, versión del modelo registrada. Sin actualización de pesos [19]. |
| **Ventajas** | Eficiencia de datos radical: opera con 15 ejemplos frente a ≈700 de la rama clásica. Multilingüe de forma nativa. Transmite un criterio de codificación propio mediante definiciones y ejemplos, lo que el *zero-shot* no logra [20], [21]. Puede devolver una alternativa y una justificación en lenguaje natural, útil para el codificador. Evidencia de acuerdo con humanos comparable o superior al de anotadores entrenados a bajo costo [22], [23]. |
| **Desventajas** | Evidencia contradictoria: afinar modelos pequeños sigue superando al *few-shot* en 16 conjuntos [24]; el desempeño depende del número de ejemplos con óptimo entre 5 y 20 y decae después (*over-prompting*) [25]. Dependencia de un proveedor externo, versión del modelo y estabilidad entre ejecuciones; costo por llamada; sin probabilidad nativa (la confianza se aproxima con la consistencia entre ejecuciones y la existencia de segunda opción). Riesgo de contaminar al codificador humano si ve las sugerencias antes de codificar [26]. |
| **Complejidad** | Sin entrenamiento. Inferencia O($L_prompt^2 \cdot d$) por llamada en el proveedor, unos 4 000 tokens de entrada por norma; costo estimado por debajo de USD 5 por pasada completa del corpus; una pasada de 813 normas en minutos. |
| **Casos de uso típicos** | Anotación asistida en ciencias sociales computacionales, codificación deductiva, clasificación de política pública con taxonomías propias [22], [27], [28]. |
| **Veredicto** | **Seleccionada como rama de comparación. Es la técnica cuya eficiencia de datos decide si el observatorio puede automatizar sin financiar miles de horas de etiquetado, y la única forma honesta de saber si supera al lineal es medirlo sobre idéntico conjunto de prueba.** |

## 3.8 Extra: enfoques basados en agentes

La ruta de actividades sugiere explorar, si el problema lo amerita, un enfoque basado en agentes —un LLM que razona por pasos, decide qué herramientas invocar (leer la norma completa en su URL oficial, consultar las definiciones del framework, buscar normas similares ya codificadas) y verifica su propia respuesta antes de emitirla [29], [30]— como alternativa a un modelo predictivo tradicional.

| | |
|---|---|
| **Cómo se aplicaría** | Un agente por norma con tres herramientas: (1) recuperar el texto completo desde la URL oficial cuando el resumen sea insuficiente; (2) recuperar las 3-5 normas ya codificadas más similares (búsqueda por embeddings) como evidencia; (3) verificar que la dimensión propuesta satisface la definición del framework y, en caso contrario, reconsiderar. Salida: dimensión, alternativa, evidencia citada y justificación. |
| **Ventajas** | Puede resolver los casos que el resumen no permite decidir accediendo a la fuente; produce una justificación con evidencia trazable, lo que interesa a los codificadores; la recuperación de normas similares aporta el criterio del equipo sin etiquetar más; se acomoda de forma natural al flujo de operación con revisión humana. |
| **Desventajas** | Coste por norma de 5 a 20 veces superior al *few-shot* simple (varias llamadas, textos completos); latencia de decenas de segundos; no determinista y difícil de evaluar de forma pareada, porque el resultado depende de qué herramientas invocó; las URL oficiales pueden estar caídas o ser PDF escaneados; complejidad de ingeniería que no cabe en las siete semanas. La |

| | |
|---|---|
| | evaluación con κ y McNemar exige salidas comparables, que un agente no garantiza. |
| **Complejidad** | Inferencia O(pasos · $L^2 \cdot d$) con $L$ que puede llegar a decenas de miles de tokens si se lee la norma completa; costo por pasada del corpus de decenas de dólares; sin entrenamiento. |
| **Casos de uso típicos** | Asistentes de investigación jurídica, revisión documental con verificación de fuentes, flujos de codificación asistida donde el humano decide con la evidencia recuperada. |
| **Veredicto** | No como sustituto del modelo predictivo en este ciclo: el problema, con textos resumidos de 44 palabras y una taxonomía cerrada de cinco clases, no amerita la complejidad ni el costo. Sí como extensión natural del flujo de operación (Figura 2 del documento técnico): las normas enrutadas a revisión humana por baja confianza podrían llegar al codificador con evidencia recuperada por un agente. Se registra como trabajo futuro con esa función acotada. |

## 4. Tabla comparativa consolidada

**Tabla 2.** Comparación de las ocho técnicas frente a los criterios de selección. ● favorable · ◐ intermedio · ○ desfavorable.

| Técnica | Ajuste al tipo de problema (texto corto, 5 clases) | Datos: rinde con ≈700 ejemplos | Recursos: CPU, 7 semanas | Interpretabilidad para el codificador | Multilingüe | Probabilidad / alternativa (top-2) | Evidencia en dominio legal-política |
|---|---|---|---|---|---|---|---|
| **Regresión Logística + TF-IDF** | ● | ● | ● | ● | ○ | ● | ◐ |
| **SVM lineal + TF-IDF** | ● | ● | ● | ● | ○ | ◐ | ● |
| **Random Forest** | ◐ | ◐ | ● | ◐ | ○ | ● | ○ |
| **Gradient Boosting / LightGBM** | ◐ | ◐ | ● | ◐ | ○ | ● | ○ |
| **fastText / CNN sobre embeddings** | ● | ○ | ◐ | ○ | ◐ | ● | ◐ |
| **Transformer afinado (BETO, XLM-R, Legal-BERT)** | ● | ○ | ○ | ○ | ● | ● | ● |
| **LLM few-shot** | ● | ● | ◐ | ◐ | ● | ◐ | ◐ |
| **Agente basado en LLM (extra)** | ◐ | ● | ○ | ● | ● | ◐ | ○ |

**Tabla 3.** Complejidad computacional y costo estimado en este corpus (813 normas).

| Técnica | Entrenamiento | Inferencia por norma | Costo estimado en este proyecto |
|---|---|---|---|
| **Regresión Logística + TF-IDF** | O(n·d·k) por iteración | O(d·k) | Segundos en CPU; USD 0 |
| **SVM lineal** | O(n·d) por iteración | O(d·k) | Segundos en CPU; USD 0 |
| **Random Forest** | O(T·n·log n·√d) | O(T·prof.) | Minutos en CPU; USD 0 |
| **Gradient Boosting** | O(T·n·d) (histogramas) | O(T·prof.) | Minutos en CPU + búsqueda de hiperparámetros; USD 0 |
| **fastText / CNN** | O(épocas·n·L·d·f) | O(L·d·f) | Minutos (fastText) a horas (CNN) en CPU; USD 0 |
| **Transformer afinado** | O(épocas·n·$L^2$·d·capas) | O($L^2$·d·capas) | Horas en GPU o inviable en CPU; acceso a modelos no garantizado |
| **LLM few-shot** | — | O($L_prompt^2$·d) en el proveedor | ≈1 300 llamadas (prueba × 3 + pasada completa); < USD 5 por pasada |
| **Agente LLM** | — | O(pasos·$L^2$·d), L hasta decenas de miles | Decenas de USD por pasada; latencia de decenas de segundos por norma |

## 5. Matriz de decisión ponderada

Los pesos reflejan los criterios de la tutoría y las restricciones del proyecto: el ajuste al problema y la interpretabilidad pesan más porque son los requisitos de los stakeholders; el multilingüismo pesa menos tras el hallazgo del EDA de que las observaciones están en español. Escala 1-5 por criterio.

**Tabla 4.** Matriz de decisión ponderada (pesos: problema 20 %, datos 20 %, recursos 15 %, interpretabilidad 20 %, multilingüe 10 %, evidencia 15 %).

| Técnica | Problema (0.20) | Datos (0.20) | Recursos (0.15) | Interpret. (0.20) | Multilingüe (0.10) | Evidencia (0.15) | Total |
|---|---|---|---|---|---|---|---|
| **Regresión Logística + TF-IDF** | **5** | **5** | **5** | **5** | **2** | **3** | **4.40** |
| **SVM lineal + TF-IDF** | **5** | **5** | **5** | **4** | **2** | **4** | **4.35** |
| **LLM few-shot** | 5 | 5 | 3 | 3 | 5 | 3 | 4.00 |
| **Agente LLM (extra)** | 3 | 5 | 1 | 4 | 5 | 2 | 3.35 |
| **Transformer afinado** | 5 | 2 | 1 | 2 | 5 | 5 | 3.20 |
| **Random Forest** | 3 | 3 | 5 | 3 | 2 | 2 | 3.05 |
| **Gradient Boosting / LightGBM** | 3 | 3 | 4 | 3 | 2 | 2 | 2.90 |
| **fastText / CNN** | 4 | 2 | 3 | 2 | 3 | 3 | 2.80 |

Las dos técnicas mejor puntuadas son también las dos de naturaleza más contrastante: un modelo lineal que aprende de ≈700 ejemplos y un modelo de lenguaje que opera con 15. Esa complementariedad —y no solo el puntaje— es lo que las convierte en el par correcto para un experimento pareado: si un observatorio con recursos limitados puede automatizar, la respuesta depende de cuál de las dos gana y por cuánto. La SVM lineal, empatada en la práctica con la regresión logística, se conserva como ablación; los transformers afinados, con la mejor evidencia de desempeño pero la peor relación con los datos y los recursos disponibles, quedan como la técnica a considerar cuando el corpus crezca.

## 6. Conclusión: de la comparación a la decisión

**Decisión que alimenta la Ficha de Decisión Técnica**

Se seleccionan **dos técnicas** para compararlas sobre idéntico conjunto de prueba a ciegas: **Regresión Logística sobre TF-IDF** (rama clásica y línea base, interpretable, probabilística, sin costo) y **LLM en modo few-shot** (rama de comparación, eficiente en datos, multilingüe, con alternativa y justificación). Ambas se miden contra dos líneas base sin IA —clase mayoritaria y metadata— y contra el acuerdo humano-humano. La SVM lineal se ejecuta como ablación de la rama clásica; los transformers afinados y el enfoque basado en agentes se documentan como trabajo futuro con funciones acotadas (mayor volumen de datos y asistencia con evidencia en la revisión humana, respectivamente).

## Referencias


[1] scikit-learn developers, “Choosing the right estimator,” scikit-learn 1.8 documentation, 2026. [Online]. Available: https://scikit-learn.org/stable/machine_learning_map.html (accedido: 20 sep. 2026).

[2] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in Proc. 22nd Int. Conf. on Machine Learning (ICML), Bonn, Germany, 2005, pp. 625–632, doi: 10.1145/1102351.1102430.

[3] S. Wang and C. D. Manning, “Baselines and bigrams: Simple, good sentiment and topic classification,” in Proc. 50th Annu. Meeting of the Association for Computational Linguistics (ACL), vol. 2, Jeju Island, Korea, 2012, pp. 90–94.

[4] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. Katz, and N. Aletras, “LexGLUE: A benchmark dataset for legal language understanding in English,” in Proc. 60th Annu. Meeting of the Association for Computational Linguistics (ACL), vol. 1, Dublin, Ireland, May 2022, pp. 4310–4330, doi: 10.18653/v1/2022.acl-long.297.

[5] J. Lee, “Legal text classification in Korean sexual offense cases: From traditional machine learning to large language models with XAI insights,” Artificial Intelligence and Law, 2025, doi: 10.1007/s10506-025-09454-w.

[6] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995, doi: 10.1007/BF00994018.

[7] T. Joachims, “Text categorization with support vector machines: Learning with many relevant features,” in Proc. 10th European Conf. on Machine Learning (ECML), Chemnitz, Germany, 1998, pp. 137–142, doi: 10.1007/BFb0026683.

[8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[10] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 2017, pp. 3146–3154.

[11] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, “Bag of tricks for efficient text classification,” in Proc. 15th Conf. of the European Chapter of the Association for Computational Linguistics (EACL), vol. 2, Valencia, Spain, 2017, pp. 427–431, doi: 10.18653/v1/E17-2068.

[12] Y. Kim, “Convolutional neural networks for sentence classification,” in Proc. 2014 Conf. on Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, 2014, pp. 1746–1751, doi: 10.3115/v1/D14-1181.

[13] J. Cañete, G. Chaperon, R. Fuentes, J.-H. Ho, H. Kang, and J. Pérez, “Spanish pre-trained BERT model and evaluation data,” in Practical ML for Developing Countries Workshop, ICLR 2020, 2020.

[14] A. Conneau et al., “Unsupervised cross-lingual representation learning at scale,” in Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 8440–8451, doi: 10.18653/v1/2020.acl-main.747.

[15] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The muppets straight out of law school,” in Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 2898–2904, doi: 10.18653/v1/2020.findings-emnlp.261.

[16] I. Chalkidis, M. Fergadiotis, and I. Androutsopoulos, “MultiEURLEX – A multi-lingual and multi-label legal document classification dataset for zero-shot cross-lingual transfer,” in Proc. 2021 Conf. on Empirical Methods in Natural Language Processing (EMNLP), Punta Cana, Dominican Republic, Nov. 2021, pp. 6974–6996, doi: 10.18653/v1/2021.emnlp-main.559.

[17] A. Vaswani et al., “Attention is all you need,” in Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 2017, pp. 5998–6008.

[18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. 2019 Conf. of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT), Minneapolis, MN, USA, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.

[19] T. B. Brown et al., “Language models are few-shot learners,” in Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020, pp. 1877–1901.

[20] C. Tang, Z. Liu, C. Ma, Z. Wu, Y. Li, W. Liu, D. Zhu, Q. Li, X. Li, T. Liu, and L. Fan, “PolicyGPT: Automated analysis of privacy policies with large language models,” arXiv:2309.10238 [cs.CL], Sep. 2023.

[21] I. Chalkidis, “ChatGPT may pass the bar exam soon, but has a long way to go for the LexGLUE benchmark,” arXiv:2304.12202 [cs.CL], Apr. 2023, doi: 10.2139/ssrn.4385460.

[22] F. Gilardi, M. Alizadeh, and M. Kubli, “ChatGPT outperforms crowd workers for text-annotation tasks,” Proceedings of the National Academy of Sciences, vol. 120, no. 30, art. e2305016120, 2023, doi: 10.1073/pnas.2305016120.

[23] E. Gunes and C. K. Florczak, “Replacing or enhancing the human coder? Multiclass classification of policy documents with large language models,” Journal of Computational Social Science, vol. 8, no. 2, art. 31, 2025, doi: 10.1007/s42001-025-00362-2.

[24] A. Edwards and J. Camacho-Collados, “Language models for text classification: Is in-context learning enough?,” in Proc. 2024 Joint Int. Conf. on Computational Linguistics, Language Resources and Evaluation (LREC-COLING), Torino, Italy, May 2024. arXiv:2403.17661.

[25] Y. Tang, D. Tuncel, C. Koerner, and T. Runkler, “The few-shot dilemma: Over-prompting large language models,” arXiv:2509.13196 [cs.CL], Sep. 2025.

[26] H. Schroeder, D. Roy, and J. Kabbara, “Just put a human in the loop? Investigating LLM-assisted annotation for subjective tasks,” in Findings of the Association for Computational Linguistics: ACL 2025, Vienna, Austria, Jul. 2025, pp. 25771–25795, doi: 10.18653/v1/2025.findings-acl.1323.

[27] R. Chew, J. Bollenbacher, M. Wenger, J. Speer, and A. Kim, “LLM-assisted content analysis: Using large language models to support deductive coding,” arXiv:2306.14924 [cs.CL], Jun. 2023.

[28] C. Ziems, W. Held, O. Shaikh, J. Chen, Z. Zhang, and D. Yang, “Can large language models transform computational social science?,” Computational Linguistics, vol. 50, no. 1, pp. 237–291, 2024, doi: 10.1162/coli_a_00502.

[29] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in Proc. 11th Int. Conf. on Learning Representations (ICLR), Kigali, Rwanda, 2023. arXiv:2210.03629.

[30] L. Wang et al., “A survey on large language model based autonomous agents,” Frontiers of Computer Science, vol. 18, no. 6, art. 186345, 2024, doi: 10.1007/s11704-024-40231-1.